SOFTWARE PROJECT REPORT | ENGLISH

Decompile 3D

AI-assisted PCB visualization and electronics learning

A full technical overview of the project purpose, target problem, architecture, implementation, APIs, data model, user experience, setup, limitations and future development.

APPLICATION Educational web prototype	**CLIENT** React + Three.js
SERVER Python + FastAPI	**AI SERVICES** Gemini, Groq, OpenRouter

Prepared from the supplied README, source files, configuration examples and sample board data.

Date: 02 November 2026

1. Executive Summary

Decompile 3D is a browser-based educational prototype that turns a PCB photograph into an approximate component map and presents that map through an interactive 3D board, photo overlays, sample signal waveforms and an AI electronics tutor. The application combines a React frontend, a Python FastAPI backend, Three.js rendering and external generative AI services.

The primary goal is to make an unfamiliar circuit board easier for a student or hobbyist to explore. A user can inspect a sample board, upload a photo, select a part, view its estimated role, see an illustrative waveform for its component type and ask the tutor a question in natural language.

Project at a glance

Item	Current implementation
Project category	AI-assisted educational visualization web app / prototype
Frontend	React 18, Vite 5, Three.js, React Three Fiber, Drei, CSS
Backend	Python, FastAPI, Pydantic request models, HTTPX async calls
Vision provider	Gemini API: PCB photo to structured JSON estimate
Tutor providers	Groq first, Gemini fallback, optional OpenRouter fallback
Core input	PCB image, or included sample board spec
Core output	Estimated parts, interactive board view, explanations and teaching visuals
Local ports	Frontend 5173; backend 8000

Interpretation: this is a learning and demonstration tool. Its photo-based part positions, component roles, holographic routing and waveform examples are estimates or illustrations. The current code does not recover a verified electrical netlist, simulate a real circuit or measure hardware.

2. Background, Problem and Scope

Problem statement

A printed circuit board contains many small packages, reference labels and connections. For a beginner, a flat photograph can be difficult to interpret: the component type may be unclear, its location can be hard to describe, and its function in the circuit may not be obvious. Explanations are often separated from the image, so learners must switch between photos, diagrams and reference material.

Project objective

Provide one visual workspace where an AI-generated part estimate, a 3D representation, a photo overlay and a conversational explanation can be explored together. The application is intended to lower the first-step learning barrier; it is not designed to certify the identity or electrical function of a board.

Target users and primary scenarios

User	Example scenario
Electronics students	Select a resistor, IC or inductor and ask what it commonly does.
Makers and hobbyists	Explore a sample or photographed board before consulting a datasheet.
Repair learners	Use the 3D view and labels as an initial visual aid while learning board vocabulary.
Demonstrators / educators	Show how photo recognition, structured data and 3D rendering fit into one app.

In-scope capabilities

- Load a built-in demo board when the frontend starts.
- Resize and submit a user-selected image for Gemini vision analysis.
- Render approximate parts in a Three.js scene and place clickable markers over the real photo or a fallback top view.
- Select parts across 3D, photo markers and component chips; show part details and a teaching waveform.
- Send user questions, recent chat messages, board data and the selected part to an AI tutor.
- Offer light, dark and hologram display modes, flow visualization, camera presets, auto-rotation and fullscreen.

Out of scope in the current implementation

PCB CAD authoring, manufacturing file generation, schematic recovery, copper-track extraction, verified pin-to-pin connectivity, SPICE simulation, live electrical measurement, user accounts, server-side project storage and production access control are not implemented.

3. System Architecture

The project runs as two local development services. Vite serves the frontend and proxies `/api` requests to FastAPI at `localhost:8000`. FastAPI serves the sample board, reports provider-key availability and makes asynchronous HTTP calls to the configured AI services.

Photo-analysis path

1. Browser	2. React	3. FastAPI	4. Gemini	5. Frontend
Choose PCB photo	Resize + data URL	POST /api/analyze	Vision -> JSON	Render board spec

Tutor-chat path

Browser	FastAPI	Groq	Gemini	OpenRouter
Question + current context	Prompt + last 10 messages	First provider	Fallback if configured	Optional final fallback

Main architecture layers

Layer	Responsibilities
Presentation layer	React components, CSS layout, file selection, chat interaction, selection state and responsive tabs.
3D rendering layer	React Three Fiber scene graph, Three.js materials and meshes, Drei camera/label/line helpers.
API layer	FastAPI routes, Pydantic payload models, environment configuration, JSON cleanup and provider calls.
AI provider layer	Gemini vision for images; chat providers attempted in configured order.
Shared data layer	A board JSON spec connects the photo analysis result to the renderer, component list and tutor context.

4. End-to-End User Workflow

Step	Implementation	User-visible result
1. Start	App calls `GET /api/sample` in a React effect.	The included sample board loads as the initial view.
2. Select photo	A browser canvas scales the longest image edge to at most 1024 px and exports JPEG quality 0.85.	The user receives a photo-analysis status message.
3. Analyze	The frontend POSTs `{image: dataUrl}` to `/api/analyze`. Vite forwards `/api` to FastAPI.	Gemini vision is asked for a board and parts JSON object.
4. Normalize	Backend parses the JSON, cleans types and numeric fields, clamps coordinate values and validates board aspect.	The resulting spec replaces the sample in frontend state.
5. Explore	The same spec is passed to Board3D, PhotoPane, component chips, Wave and chat context.	The 3D board, photo markers, details and waveform update.
6. Ask tutor	The latest 10 messages, board spec and optional selected component are posted to `/api/chat`.	The first available configured provider returns a text answer.

State and persistence

The visible board, photo, chat, selected part, theme and flow mode are maintained in React state. The reviewed frontend does not send user boards to an application database or restore them after a page reload. The only persistent external configuration shown in the source is environment-based provider configuration.

Responsive behavior

At narrower widths the layout changes to a single-column interface. Bottom tabs switch between the 3D/photo view, waveform and tutor panels. The desktop layout presents the board/photo pair and tutor panel side by side.

5. Image Analysis and Board Data Model

Vision prompt and response

The backend asks Gemini to return JSON containing a `board` object with `aspect` and `name`, and a `parts` array. Each part can include `type`, `label`, `value`, `x`, `y`, `w`, `h` and `role`. The prompt asks for visible parts, approximate top-down positions, short role descriptions and no more than 35 parts. The cleanup function processes up to 40 returned entries.

Coordinate convention

Coordinates are percentages of the visible board bounding box. The top-left is `(0, 0)`. `x` and `y` describe the part center. `w` and `h` describe an estimated footprint size. These are visual estimates, not measured CAD dimensions.

Example data contract

{ "board": { "name": "Example board", "aspect": 1.4 },
"parts": [{ "type": "inductor", "label": "L1", "value": "2R2",
"x": 67, "y": 17, "w": 25, "h": 25,
"role": "Stores and releases energy in a switching converter." }] }

Normalization performed by `clean()`

- Unrecognized component types become `other`; the allow-list contains 16 types: USB-C, inductor, IC, resistor, capacitor, electrolytic capacitor, diode, LED, pad, hole, connector, transistor, crystal, button, antenna and other.
- Numeric strings and percent suffixes can be converted to numbers. Invalid numeric values use defaults.
- x and y are clamped to 0-100. Width and height are clamped to 0.5-100.
- Board aspect defaults to 1 when missing or outside the accepted range.
- Part text fields are converted to trimmed strings. Non-dictionary part records are skipped.

Important limitation: there is no confidence field, review queue or component-correction editor. Perspective rectification, reference-designator OCR verification and copper-track segmentation are not part of the current pipeline.

6. Frontend Design and Interaction

Component	Role
`App.jsx`	Owns application state and actions: loading sample data, resizing/uploading photos, chat, selection, theme, fullscreen and flow mode.
`PhotoPane` in `App.jsx`	Displays the original uploaded image or fallback board view with percentage-positioned click targets.
`Board3D.jsx`	Builds the 3D scene, part meshes, camera controls, labels, hologram and flow overlays.
`Wave.jsx`	Produces component-type waveform examples with a Canvas 2D animation loop.
`styles.css`	Defines desktop/mobile layouts, photo markers, chat, waveform panel and flow card.
`main.jsx`	Mounts the React app and imports the shared stylesheet.

Selection model

Selecting a part in the 3D scene, on the photo, or in the component chip list sets one shared selected-part object. The selected part details are shown in the tutor panel. Camera focus moves the view target to the selected part and disables automatic rotation. The selection also influences tutor context, waveform type and the optional selected-component flow.

Chat rendering

The frontend renders assistant replies with a small markdown-like parser for headings, lists, bold text and inline code. HTML tags are removed. Chat history is kept in component state and scrolled to the latest message.

Fullscreen

The 3D view uses the browser Fullscreen API. A `fullscreenchange` event updates the button label. In fullscreen, the board view and its overlays remain inside the expanded view element.

7. 3D Model, Hologram and Flow Rendering

Board dimensions and component geometry

The 3D board has a fixed width of 30 world units. Height is derived from the board aspect ratio and bounded to a minimum and maximum. Percentage x/y coordinates map to world x/z coordinates around the board center. Part w/h determine footprint dimensions; a type-to-height map gives generic component heights. Most components are boxes; pads and electrolytic capacitors use cylinders; mounting holes use a cylinder geometry.

Materials and selection

Standard materials use per-type colors with metalness and roughness variation. Selected components receive an emissive highlight. ICs have generic metallic leads. The current source includes generic solder pads and terminal details around many part shapes; these are visual aids, not datasheet-accurate footprints.

Hologram implementation

Hologram mode uses a custom GLSL vertex and fragment shader. It calculates a view-angle edge glow using a Fresnel-style term, animated scan bands, subtle time-based flicker and occasional hash-based glitch accents. Additional scene elements include a board grid helper, edge outlines and a moving scan beam. The material uses additive blending and transparency to create a projected cyan look.

Flow visualization

Power, return and selected-component paths are assembled from inferred positions of a USB-C/connector, IC, inductor, output pad and selected part. The code builds intermediate orthogonal waypoints and renders a smooth Catmull-Rom curve, a broad glow line, a bright core line and moving particles. The selector offers All, Power, Return and Selected modes.

Electrical meaning: flow lines do not come from an actual schematic or netlist. They are illustrative paths generated from estimated component locations; they should not be used to infer real connectivity or current direction.

8. AI Tutor and Waveform Learning Aids

Tutor behavior

The backend system prompt asks the tutor to answer in the student's language, keep simple answers short, avoid unnecessary quizzes or summaries, remove HTML and treat AI-detected board information as uncertain when appropriate. It supplies board JSON and selected-part context when present. Only the last 10 user/assistant messages are forwarded in each request.

Provider order and model handling

Purpose	Provider behavior
Photo analysis	Gemini vision endpoint; configured `GEMINI_MODEL` is tried first when present, followed by code-defined candidates.
Chat first choice	Groq chat completion endpoint. Configured `GROQ_MODEL` is tried before default candidates.
Chat fallback	Gemini chat is tried if a Gemini key is configured and Groq is absent or fails.
Optional final fallback	OpenRouter chat is tried if its key is configured and earlier providers did not return text.

Waveform generation

Waveform definitions live in `Wave.jsx`. The canvas loop uses `requestAnimationFrame` and JavaScript functions for sine waves, square waves, ramps, stepped signals and component-specific teaching patterns. Examples are associated with USB input, inductor, IC, resistor, capacitor, diode, LED, battery pad, transistor, crystal, connector, button and antenna. An NTC-style pad can receive a special example based on its label.

Signal caveat: no ADC input, oscilloscope data or SPICE engine feeds these plots. They are animated educational examples selected by component type, not measured voltage or current.

9. Backend API and Error Handling

Method and route	Purpose	Observed behavior
GET `/api/status`	Report provider key availability.	Returns booleans based on whether each environment key is non-empty; it does not validate keys.
GET `/api/sample`	Return built-in sample JSON.	Loaded from `backend/sample.json` at server startup.
POST `/api/analyze`	Convert an image into a board spec.	Requires Gemini key; provider/parse failures return JSON errors. Empty parts return 422.
POST `/api/chat`	Answer a board-aware electronics question.	Uses configured providers in order. If none returns text, it returns a warning-style reply.

HTTP and provider behavior

`httpx.AsyncClient` uses a 180-second timeout. A shared helper tries model candidates in order, returns JSON on HTTP 200 and records a short error string for failures. Authentication errors with status 401, 402 or 403 stop further model attempts for that provider. The application returns explicit JSON errors for missing Gemini key, provider errors, invalid or empty JSON and no detected parts.

Environment configuration

At startup, the backend checks `backend/.env` followed by the repository-root `.env`. The example file names `GEMINI_API_KEY`, `GROQ_API_KEY` and `OPENROUTER_API_KEY`; optional model variables are `GEMINI_MODEL`, `GROQ_MODEL` and `OPENROUTER_MODEL`. Actual key values are not included in this report.

10. Technology Stack and Repository Map

Area	Technology / file	Role
Frontend	React 18.3, Vite 5.2	Single-page application and local development server.
3D	Three.js 0.160, React Three Fiber 8.15, Drei 9.100	Scene graph, geometry, materials, camera controls and labels.
Backend	Python, FastAPI, Uvicorn	HTTP API, asynchronous provider calls and local server.
HTTP / validation	httpx, Pydantic (via FastAPI)	Provider requests and request/response data validation.
AI	Gemini vision; Groq and optional OpenRouter for chat	Image interpretation and conversational responses.
Files	`backend/main.py`	API routes, prompts, provider helpers and data cleanup.
Files	`backend/sample.json`	Initial example board specification.
Files	`frontend/App.jsx`	App-level state, upload, theme, chat and fullscreen controls.
Files	`frontend/Board3D.jsx`, `frontend/Wave.jsx`	3D scene / board flows and illustrative waveform canvas.
Files	`frontend/vite.config.js`, `run.bat`, `.gitignore`	API proxy, local launch and ignored local artifacts.

Data contract is the integration seam

The board specification is the shared hand-off between AI analysis and the visual experience. If its coordinates, dimensions or component vocabulary change, the API cleaner, 2D photo overlays, component list, 3D placement and tutor prompt may all need coordinated updates.

11. Local Setup and Operation

The repository is arranged for a local Windows development workflow. `run.bat` starts the API and Vite development server, then opens the browser at `localhost:5173`. The Vite development configuration forwards API paths to `localhost:8000`. Install backend and frontend dependencies before launch and provide provider keys in an ignored `.env` file.

Typical demonstration sequence

- Launch the app and show the included sample board.
- Rotate and zoom the 3D board; select a part and show synchronized selection.
- Switch Light, Dark and Hologram themes; demonstrate available flow overlays.
- Open Fullscreen and inspect the side information/flow display.
- Upload a clear top-down PCB photo and explain that the AI returns an estimated board spec.
- Ask the tutor what a selected component usually does; clarify that answers are AI-generated.

Launcher implementation note

The batch launcher searches process output for strings containing `:8000` and `:5173` and then force-terminates matching process IDs before starting services. On a developer machine this could match another local process or a port substring, so inspect the script and close services deliberately if the machine hosts other development apps. This is a local convenience script, not a production service manager.

12. Limitations, Security and Readiness

Area	Current limitation / implication
Image understanding	AI estimates component types and positions from pixels; occlusion, blur, perspective, glare and silkscreen ambiguity can reduce accuracy.
Coordinates and geometry	The 3D view is a schematic visualization based on normalized estimates, not a trace-accurate CAD reconstruction.
Electrical connectivity	Flow overlays are illustrative paths inferred from component positions and categories; there is no extracted netlist or verified current calculation.
Waveform	Waveforms are animated examples, not simulated or measured electrical signals.
AI answers	Provider output may be incorrect or overly confident; treat explanations as educational guidance and verify against datasheets.
Web security	CORS currently allows all origins, methods and headers; that is permissive for a local prototype.
Secrets	Keys are read from local environment files and ignored by Git; keep keys out of source control and logs.
Operations	No authentication, user accounts, durable project storage, rate limiting, observability or deployment controls are evident in the reviewed code.
Quality evidence	This report is based on source review. No runtime build, live provider call, hardware measurement or full application test was run for this document.

Before a public deployment

- Restrict CORS to approved origins and add authentication / rate controls as required.
- Add schema versioning, image-size validation, request limits and safe error reporting.
- Add user review/editing for detected parts and make uncertainty visible.
- Keep provider credentials server-side; use a managed secret store in hosted environments.
- Establish automated checks and manual validation on representative PCB photos.

13. Future Roadmap and Potential Uses

Phase	Suggested work	Value
1. Trustworthy capture	Show confidence / uncertainty, editable parts and image crop or perspective correction.	Users can correct AI estimates before interpreting the view.
2. Real board structure	Extract copper traces and pads with computer vision; build a connectivity graph and explicit nets.	More faithful structure and explainable path display.
3. Electrical simulation	Add circuit/netlist input and integrate a simulator such as SPICE where appropriate.	Waveforms and current paths can be grounded in circuit models.
4. Rich 3D assets	Use PCB/CAD formats, component footprints, connector pins and realistic board layers.	Closer-to-real geometry, scale and joint placement.
5. Project workflow	Save/export board specs and annotations; compare versions and support reproducible sessions.	Useful for documentation, review and classroom exercises.
6. Production readiness	Authentication, deployment configuration, logging, privacy controls, quotas and monitoring.	Safer shared use and operational visibility.

Potential audiences

The concept can support electronics learning, introductory PCB familiarization, design-review conversations and early visual inspection. Any repair, safety or manufacturing decision should use verified schematics, board files, component datasheets and appropriate instruments; this prototype alone does not establish electrical correctness.

14. Conclusion and Demo Narrative

Decompile 3D combines AI-assisted image interpretation with a browser-based interactive board visualization. Its central idea is to make a PCB easier to explore: inspect a sample or uploaded image, see a component-oriented 3D view, switch visual themes, explore illustrative flows, and ask a context-aware tutor for explanations.

The current implementation is best described as an educational visualization prototype. Its strongest practical next step is to make the detected board editable and expose uncertainty; its most important technical leap is deriving actual traces and connectivity so that board paths and electrical behavior can be tied to evidence.

Short presentation script

“This project takes a top-view PCB image and uses a vision model to estimate the board and its components. The frontend turns that structured result into a synchronized photo and interactive Three.js view. Users can inspect individual parts, switch to a hologram theme, view illustrative flow overlays, and ask an AI tutor questions. The current flows and waveforms are visual teaching aids; they are not measurements or circuit simulation. The roadmap is to add confidence and editing first, then trace extraction, connectivity and grounded simulation.”

Basis and scope of this report

Prepared from the project README and source files available for review: backend API and sample data, React application and 3D/waveform components, frontend package manifest and Vite configuration, requirements, launcher and Git ignore rules. No private environment values were read into the report. No external claims are required to understand this code-based description.

Report prepared 2 November 2026. Version reflects the repository contents reviewed for this report.